17. Idiomatic Go: Effective Patterns

17. Idiomatic Go: Effective Patterns

Phase: Phase 3 — Idioms and engineering judgment
Estimated time: 1–2 days
Slug: `17-idiomatic-go`

Learning objectives

- Apply Effective Go naming and package heuristics
- Make the zero value useful
- Keep interfaces small and local
- Write clear Go that reviewers trust

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Names

- Packages: short, lowercase, no underscores (`http`, `json`)
- Interfaces: often `-er` (`Reader`) when one method
- Avoid stuttering: `http.Server` not `http.HTTPServer` in the package name sense

Useful zero values

`var buf bytes.Buffer` works immediately. Design types so `var t T` is ready or clearly invalid.

Return early

Flatten happy path; handle errors first. Avoid deep nesting.

Don’t communicate by sharing memory…

…share memory by communicating (channels) — but also learn mutexes (later). Idiomatic means choosing the right tool, not dogma.

Common mistakes (especially from Python instincts)

- Java-style getters for every field
- Huge package “utils” dumping grounds
- Interface pollution (exporting interfaces no one implements outside)

Exercises

Work in `exercises/17-idiomatic-go/`. Read that folder’s `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.

- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/17-idiomatic-go/` until you have an honest failing attempt.

Self-check

1. What is a useful zero value example?
2. Why short package names?
3. When is stuttering bad?

Further reading

- https://go.dev/doc/effective_go
- https://go.dev/wiki/CodeReviewComments